

MERN STACK

DAY – 21

18 July 2026

MongoDB

MongoDB Inc. MongoDB is a NoSQL, document-oriented database used to store large amounts of data in a flexible format. Unlike traditional relational databases, MongoDB stores data as BSON (Binary JSON) documents instead of rows and columns.

Features of MongoDB

- Document-oriented: Stores data in JSON-like documents.
- Schema-less: Documents in the same collection can have different fields.
- High Performance: Fast read and write operations.
- Scalable: Supports horizontal scaling through sharding.
- High Availability: Replica sets provide automatic failover.
- Indexing: Improves query performance.
- Aggregation Framework: Performs complex data processing and analytics.

MongoDB Installation

On Windows

1. Visit the official MongoDB download page:
 - MongoDB Community Server Downloads
2. Download the MongoDB Community Server installer.
3. Run the installer.
4. Select Complete Installation.
5. Choose Install MongoDB as a Service (recommended).
6. Complete the installation.
7. Install MongoDB Compass (GUI tool) if prompted.
8. Verify installation by opening Command Prompt and running:
9. `mongod --version`

or

`mongosh`

MongoDB Backend Setup

A backend application built with Node.js, Express.js, and MongoDB allows you to create APIs that can communicate with a database. The Mongoose library helps connect Node.js with MongoDB, while CORS enables communication between the backend and frontend running on different ports.

Step 1: Create a Backend Folder

Create a separate folder for your backend project.

```
mkdir backend  
cd backend
```

Step 2: Initialize the Node.js Project

Run the following command:

```
npm init -y
```

- npm stands for Node Package Manager.
- It is automatically installed with Node.js.
- npm is used to install and manage packages (libraries).

npm init initializes a Node.js project by creating a file named package.json.

Step 3: Install Required Packages

```
npm install express mongoose cors
```

- Express is a lightweight Node.js framework used to build web servers and RESTful APIs.
- Mongoose is an Object Data Modeling (ODM) library that provides a structured way to interact with MongoDB using schemas and models.
- CORS (Cross-Origin Resource Sharing) is a browser security mechanism that controls whether a web application can make requests to a server hosted on a different origin.

Step 4: Server.js

```
const mongoose=require("mongoose")  
const express=require("express")
```

```
const cors=require("cors")
const app=express()
app.use(cors())
app.use(express.json())
app.listen(9000,()=>{
  console.log("connected")
});
mongoose.connect("mongodb://127.0.0.1:27017/newcrud").then(()=>{
  console.log("DB CONNECTED")
}).catch((err)=>{
  console.log(err);
})
```

- `const mongoose = require("mongoose");`
Used to connect Node.js with MongoDB and perform database operations.
- `const express = require("express");`
Used to create the server and REST APIs.
- `const cors = require("cors");`
Allows communication between frontend and backend running on different origins.
- `const app = express();`
Creates an Express application.
- `app.use(cors());`
Enables Cross-Origin Resource Sharing (CORS).
- `app.use(express.json());`
Parses incoming JSON data.
Makes request data available in `req.body`.
- `app.listen(9000, () => { ... });`
Starts the server on port 9000.
- `mongoose.connect("mongodb://127.0.0.1:27017/newcrud")`

Connects the application to the newcrud MongoDB database.

- `.then(() => { console.log("DB CONNECTED"); })`
Executes if the database connection is successful.
- `.catch((err) => { console.log(err); })`
Handles database connection errors.

